

PyTorch Tutorial

Shicheng Wen

Outline

- Introduction
- Tensors
- Autograd
- Module
- Dataset & DataLoader
- Tips on Implementation

Introduction



- A Python-based scientific computing package
- Supports GPU acceleration and works like NumPy
- A flexible and fast platform for deep learning research

Installation

- Follow instructions in <https://pytorch.org/get-started/locally/>

PyTorch Build	Stable (2.10.0)			Preview (Nightly)	
Your OS	Linux		Mac		Windows
Package	Pip		LibTorch		Source
Language	Python			C++ / Java	
Compute Platform	CUDA 12.6	CUDA 12.8	CUDA 13.0	ROCm 7.1	CPU
Run this Command:	pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu126				

PyTorch Hierarchy

Three Levels of Abstraction

- Tensors
- Autograd
- Module

Tensors

Tensors

- A multi-dimensional array containing elements of a single data type.
- Tensors are just like numpy arrays, but they can run on GPU.
- Storage: Points to a contiguous (or strided) block of memory.
- Metadata: Shape, Dtype, Stride, Device.
- Device Agnostic: Runs on CPU, CUDA

Tensors

- Example of a two-layer neural network using PyTorch Tensors

```
import torch
```

```
dtype = torch.float32
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
N, D_in, H, D_out = 64, 1000, 100, 10
```

```
x = torch.randn(N, D_in, device=device, dtype=dtype)
```

```
y = torch.randn(N, D_out, device=device, dtype=dtype)
```

```
w1 = torch.randn(D_in, H, device=device, dtype=dtype)
```

```
w2 = torch.randn(H, D_out, device=device, dtype=dtype)
```

```
learning_rate = 1e-6
```

```
for t in range(500):
```

```
    # Forward Pass
```

```
    h = x @ w1
```

```
    h_relu = h.clamp(min=0)
```

```
    y_pred = h_relu @ w2
```

```
    # Compute Loss
```

```
    loss = (y_pred - y).pow(2).sum().item()
```

```
    # Backward Pass
```

```
    grad_y_pred = 2.0 * (y_pred - y)
```

```
    grad_w2 = h_relu.t() @ grad_y_pred
```

```
    grad_h_relu = grad_y_pred @ w2.t()
```

```
    grad_h = grad_h_relu.clone()
```

```
    grad_h[h < 0] = 0
```

```
    grad_w1 = x.t() @ grad_h
```

```
    # Update Weights
```

```
    w1 -= learning_rate * grad_w1
```

```
    w2 -= learning_rate * grad_w2
```


Tensors

- Create random tensor for data and weight on CUDA (if available) in float32
- Tensor can also be loaded by:
 1. Load from data (list)
`data = torch.tensor([[1, 2], [3, 4]])`
 2. From Numpy array
`np_array = np.array([[1, 2], [3, 4]])`
`data = torch.from_numpy(np_array)`
 3. From another tensor
`old_tensor = torch.tensor([[1, 2], [3, 4]])`
`new_tensor = old_tensor.clone()`
Or
`new_tensor = old_tensor.detach().clone()`

```
import torch
```

```
dtype = torch.float32  
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
N, D_in, H, D_out = 64, 1000, 100, 10
```

```
x = torch.randn(N, D_in, device=device, dtype=dtype)  
y = torch.randn(N, D_out, device=device, dtype=dtype)
```

```
w1 = torch.randn(D_in, H, device=device, dtype=dtype)  
w2 = torch.randn(H, D_out, device=device, dtype=dtype)
```

```
learning_rate = 1e-6
```

```
for t in range(500):  
    # Forward Pass  
    h = x @ w1  
    h_relu = h.clamp(min=0)  
    y_pred = h_relu @ w2  
  
    # Compute Loss  
    loss = (y_pred - y).pow(2).sum().item()  
  
    # Backward Pass  
    grad_y_pred = 2.0 * (y_pred - y)  
    grad_w2 = h_relu.t() @ grad_y_pred  
    grad_h_relu = grad_y_pred @ w2.t()  
    grad_h = grad_h_relu.clone()  
    grad_h[h < 0] = 0  
    grad_w1 = x.t() @ grad_h  
  
    # Update Weights  
    w1 -= learning_rate * grad_w1  
    w2 -= learning_rate * grad_w2
```

Tensors

- Forward pass: compute predictions and loss
- @ for matrix multiplication
- Other ways:
 - torch.matmul(a, b) or a.matmul(b)
 - torch.mm(a, b) or a.mm(b) 2D only
 - torch.bmm(a, b) or a.bmm(b) 3D only
 - torch.einsum("ij,jk->ik", a, b)
 - torch.einsum("bij,bjk->bik", a, b)

```
import torch
```

```
dtype = torch.float32
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
N, D_in, H, D_out = 64, 1000, 100, 10
```

```
x = torch.randn(N, D_in, device=device, dtype=dtype)
```

```
y = torch.randn(N, D_out, device=device, dtype=dtype)
```

```
w1 = torch.randn(D_in, H, device=device, dtype=dtype)
```

```
w2 = torch.randn(H, D_out, device=device, dtype=dtype)
```

```
learning_rate = 1e-6
```

```
for t in range(500):
```

```
    # Forward Pass
```

```
    h = x @ w1
```

```
    h_relu = h.clamp(min=0)
```

```
    y_pred = h_relu @ w2
```

```
    # Compute Loss
```

```
    loss = (y_pred - y).pow(2).sum().item()
```

```
    # Backward Pass
```

```
    grad_y_pred = 2.0 * (y_pred - y)
```

```
    grad_w2 = h_relu.t() @ grad_y_pred
```

```
    grad_h_relu = grad_y_pred @ w2.t()
```

```
    grad_h = grad_h_relu.clone()
```

```
    grad_h[h < 0] = 0
```

```
    grad_w1 = x.t() @ grad_h
```

```
    # Update Weights
```

```
    w1 -= learning_rate * grad_w1
```

```
    w2 -= learning_rate * grad_w2
```

Tensors

- Backward pass: manually compute gradients if you don't have autograd.

```
import torch

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

w1 = torch.randn(D_in, H, device=device, dtype=dtype)
w2 = torch.randn(H, D_out, device=device, dtype=dtype)

learning_rate = 1e-6

for t in range(500):
    # Forward Pass
    h = x @ w1
    h_relu = h.clamp(min=0)
    y_pred = h_relu @ w2

    # Compute Loss
    loss = (y_pred - y).pow(2).sum().item()

    # Backward Pass
    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = h_relu.t() @ grad_y_pred
    grad_h_relu = grad_y_pred @ w2.t()
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t() @ grad_h

    # Update Weights
    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```

Tensors

- Optimization: Gradient descent step on weights

```
import torch

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

w1 = torch.randn(D_in, H, device=device, dtype=dtype)
w2 = torch.randn(H, D_out, device=device, dtype=dtype)

learning_rate = 1e-6

for t in range(500):
    # Forward Pass
    h = x @ w1
    h_relu = h.clamp(min=0)
    y_pred = h_relu @ w2

    # Compute Loss
    loss = (y_pred - y).pow(2).sum().item()

    # Backward Pass
    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = h_relu.t() @ grad_y_pred
    grad_h_relu = grad_y_pred @ w2.t()
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t() @ grad_h

    # Update Weights
    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```

Autograd

Gradients

- The previous process:
 - Slow
 - Gradient is hard to compute when model becomes more complex

Key:

Set `required_grad` to `True` to enable `torch.autograd`

Autograd

- A Tensor state that records operation history.
- Mechanism:
 - `requires_grad=True`: Tells PyTorch to begin tracking operations.
 - `grad_fn`: Every operation attaches a function to the output tensor, pointing back to input tensors.
- Dynamic Computational Graph

Autograd

- We set the weights' `requires_grad` to be `True`

```
import torch

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available()
                        else "cpu")
N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

w1 = torch.randn(D_in, H, device=device,
                  dtype=dtype, requires_grad=True)
w2 = torch.randn(H, D_out, device=device,
                  dtype=dtype, requires_grad=True)

learning_rate = 1e-6

for t in range(500):
    # Forward Pass
    h = x @ w1
    h_relu = h.clamp(min=0)
    y_pred = h_relu @ w2

    # Compute Loss
    loss = (y_pred - y).pow(2).sum()

    # Backward Pass
    loss.backward()

    # Update Weights
    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad

        w1.grad.zero_()
        w2.grad.zero_()
```


Autograd

- Forward pass looks exactly the same as the non-autograd version.

```
import torch

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available()
                        else "cpu")
N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

w1 = torch.randn(D_in, H, device=device,
                  dtype=dtype, requires_grad=True)
w2 = torch.randn(H, D_out, device=device,
                  dtype=dtype, requires_grad=True)

learning_rate = 1e-6

for t in range(500):
    # Forward Pass
    h = x @ w1
    h_relu = h.clamp(min=0)
    y_pred = h_relu @ w2

    # Compute Loss
    loss = (y_pred - y).pow(2).sum()

    # Backward Pass
    loss.backward()

    # Update Weights
    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad

    w1.grad.zero_()
    w2.grad.zero_()
```

Autograd

- But the gradient of loss with respect to w_1 and w_2 can be done by a simple one-line code.

```
import torch

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available()
                       else "cpu")
N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

w1 = torch.randn(D_in, H, device=device,
                  dtype=dtype, requires_grad=True)
w2 = torch.randn(H, D_out, device=device,
                  dtype=dtype, requires_grad=True)

learning_rate = 1e-6

for t in range(500):
    # Forward Pass
    h = x @ w1
    h_relu = h.clamp(min=0)
    y_pred = h_relu @ w2

    # Compute Loss
    loss = (y_pred - y).pow(2).sum()

    # Backward Pass
    loss.backward()

    # Update Weights
    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad

    w1.grad.zero_()
    w2.grad.zero_()
```

Autograd

- Make gradient step on weights.
- What's `torch.no_grad()`?
 - We need to use `NO_GRAD` to keep the update out of the gradient computation
 - Why is that? It boils down to the DYNAMIC GRAPH that PyTorch uses.
- We zero out gradients to prevent them from previous iterations from contaminating the current update.

```
import torch

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available()
                       else "cpu")
N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

w1 = torch.randn(D_in, H, device=device,
                  dtype=dtype, requires_grad=True)
w2 = torch.randn(H, D_out, device=device,
                  dtype=dtype, requires_grad=True)

learning_rate = 1e-6

for t in range(500):
    # Forward Pass
    h = x @ w1
    h_relu = h.clamp(min=0)
    y_pred = h_relu @ w2

    # Compute Loss
    loss = (y_pred - y).pow(2).sum()

    # Backward Pass
    loss.backward()

    # Update Weights
    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad

        w1.grad.zero_()
        w2.grad.zero_()
```

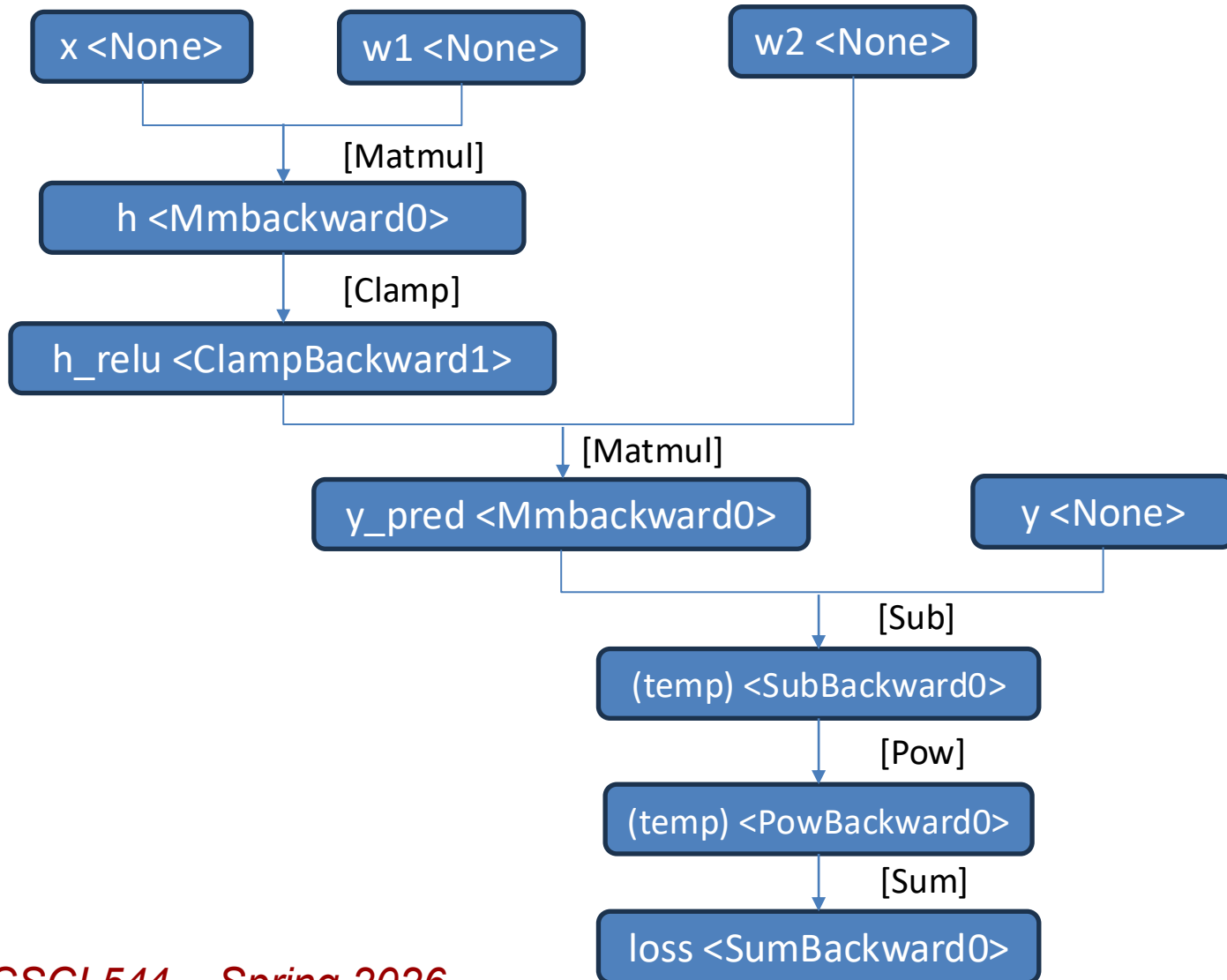
How PyTorch Remembers Operations

- Chain rule
- Dynamic Computational Graph (DAG)
 - Built dynamically forward. Destroyed backward.
 - Node: Tensors
 - Edge: Functions
- The "Define-by-Run" Paradigm:
 - PyTorch builds the graph dynamically as you execute code.

How PyTorch Remembers Operations

- Gradient Function
 - Every Tensor that is the result of an operation has a `grad_fn`.
 - It serves as a backward pointer.
- Visualizing the Chain:
 - Leaf Node (Input) $\rightarrow$ `grad_fn = None`
 - $y = x * w \rightarrow \text{grad_fn} = \langle \text{MulBackward} \rangle$
 - $z = y + b \rightarrow \text{grad_fn} = \langle \text{AddBackward} \rangle$

Graph of Gradient Function



```
import torch
```

```
dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available()
                        else "cpu")
N, D_in, H, D_out = 64, 1000, 100, 10
```

```
x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)
```

```
w1 = torch.randn(D_in, H, device=device,
                 dtype=dtype, requires_grad=True)
w2 = torch.randn(H, D_out, device=device,
                 dtype=dtype, requires_grad=True)
```

```
learning_rate = 1e-6
```

```
for t in range(500):
    # Forward Pass
    h = x @ w1
    h_relu = h.clamp(min=0)
    y_pred = h_relu @ w2
```

```
# Compute Loss
loss = (y_pred - y).pow(2).sum()
```

```
# Backward Pass
loss.backward()
```

```
# Update Weights
with torch.no_grad():
    w1 -= learning_rate * w1.grad
    w2 -= learning_rate * w2.grad
```

```
w1.grad.zero_()
w2.grad.zero_()
```

Customizing the Engine

- Why do we need custom functions?
 - Non-differentiable ops
 - Custom gradient
 - Performance
 - Research
- ctx (Context)
 - Since the local variables are lost after forward scope ends, we need a “time capsule”.
 - `ctx.save_for_backward(tensors)`: Saves data from Forward pass to be used in Backward pass.
- Structure:
 - Inherit from `torch.autograd.Function`
 - Implement static methods: `forward()` and `backward()`

New Autograd Function

- We use `torch.autograd.Function` to implement a manual ReLU
- We then apply our function in the forward pass.

```
class MyReLU(torch.autograd.Function):  
    @staticmethod  
    def forward(ctx, input_tensor):  
        ctx.save_for_backward(input_tensor)  
        return input_tensor.clamp(min=0)  
  
    @staticmethod  
    def backward(ctx, grad_output):  
        input_tensor, = ctx.saved_tensors  
  
        grad_input = grad_output.clone()  
        grad_input[input_tensor < 0] = 0  
        return grad_input
```

```
learning_rate = 1e-6  
  
for t in range(500):  
    h = x @ w1  
    h_relu = MyReLU.apply(h)  
    y_pred = h_relu @ w2  
  
    loss = (y_pred - y).pow(2).sum()  
    loss.backward()  
  
    with torch.no_grad():  
        w1 -= learning_rate * w1.grad  
        w2 -= learning_rate * w2.grad  
        w1.grad.zero_()  
        w2.grad.zero_()
```

Module

torch.nn

- The problem with raw autograd:
 - Manually managing weights is messy.
 - Hard to save/load models.
 - Hard to reuse standard layers.
- torch.nn
 - Higher-level wrapper for working with neural networks

```
import torch
import torch.nn as nn

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available()
                       else "cpu")

N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

model = nn.Sequential(
    nn.Linear(D_in, H),
    nn.ReLU(),
    nn.Linear(H, D_out),
)

model.to(device=device, dtype=dtype)

loss_fn = nn.MSELoss(reduction='sum')

learning_rate = 1e-6

for t in range(500):
    # Forward Pass
    y_pred = model(x)

    # Compute Loss
    loss = loss_fn(y_pred, y)

    # Backward Pass
    model.zero_grad()
    loss.backward()

    # Update Weights
    with torch.no_grad():
        for param in model.parameters():
            param -= learning_rate * param.grad
```

torch.nn

- Define our model as a sequence of layers
- torch.nn also defines common loss functions

```
import torch
import torch.nn as nn

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available()
                       else "cpu")

N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

model = nn.Sequential(
    nn.Linear(D_in, H),
    nn.ReLU(),
    nn.Linear(H, D_out),
)

model.to(device=device, dtype=dtype)

loss_fn = nn.MSELoss(reduction='sum')

learning_rate = 1e-6

for t in range(500):
    # Forward Pass
    y_pred = model(x)

    # Compute Loss
    loss = loss_fn(y_pred, y)

    # Backward Pass
    model.zero_grad()
    loss.backward()

    # Update Weights
    with torch.no_grad():
        for param in model.parameters():
            param -= learning_rate * param.grad
```

torch.nn

- Forward pass:
 - Feed data to model
 - Use prediction to and ground truth to get loss

```
import torch
import torch.nn as nn

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available()
                        else "cpu")

N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

model = nn.Sequential(
    nn.Linear(D_in, H),
    nn.ReLU(),
    nn.Linear(H, D_out),
)

model.to(device=device, dtype=dtype)

loss_fn = nn.MSELoss(reduction='sum')

learning_rate = 1e-6

for t in range(500):
    # Forward Pass
    y_pred = model(x)

    # Compute Loss
    loss = loss_fn(y_pred, y)

    # Backward Pass
    model.zero_grad()
    loss.backward()

    # Update Weights
    with torch.no_grad():
        for param in model.parameters():
            param -= learning_rate * param.grad
```

torch.nn

- Backward pass:
 - torch.nn handles autograd for us!
 - We can simply use `model.zero_grad()` to clear all gradients for the parameters in the model

```
import torch
import torch.nn as nn

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available()
                        else "cpu")

N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

model = nn.Sequential(
    nn.Linear(D_in, H),
    nn.ReLU(),
    nn.Linear(H, D_out),
)

model.to(device=device, dtype=dtype)

loss_fn = nn.MSELoss(reduction='sum')

learning_rate = 1e-6

for t in range(500):
    # Forward Pass
    y_pred = model(x)

    # Compute Loss
    loss = loss_fn(y_pred, y)

    # Backward Pass
    model.zero_grad()
    loss.backward()

    # Update Weights
    with torch.no_grad():
        for param in model.parameters():
            param -= learning_rate * param.grad
```

torch.nn

- Update weights using gradient descent on each model parameter.
- Question:
 - How can we apply more advanced rules for updating?
 - Any easier way?

```
import torch
import torch.nn as nn

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available()
                       else "cpu")

N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

model = nn.Sequential(
    nn.Linear(D_in, H),
    nn.ReLU(),
    nn.Linear(H, D_out),
)

model.to(device=device, dtype=dtype)

loss_fn = nn.MSELoss(reduction='sum')

learning_rate = 1e-6

for t in range(500):
    # Forward Pass
    y_pred = model(x)

    # Compute Loss
    loss = loss_fn(y_pred, y)

    # Backward Pass
    model.zero_grad()
    loss.backward()

    # Update Weights
    with torch.no_grad():
        for param in model.parameters():
            param -= learning_rate * param.grad
```

Optimizer

- A simple way
 - Call `torch.optim` package, which contains various advanced optimizer other than SGD.
 - Now, all the parameters can be updated via one-line code.
- Inside `optimizer.step()`
 - Iteration:
 - It loops over all parameters groups provided during initialization.
 - State Update (Adam Example):
 - It updates the "First Moment" (Mean of gradients).
 - It updates the "Second Moment" (Uncentered variance).
 - Parameter Update:
 - It modifies `param.data` in-place using the calculated step.

```
learning_rate = 1e-4
optimizer = torch.optim.Adam(model.parameters(),
                               lr=learning_rate)

for t in range(500):
    # Forward Pass
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    # Backward Pass
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Define New Modules

- PyTorch Module is a neural network layer, it can contain weights or other modules.
- It provides a more systematic way to structure our code.

```
import torch
import torch.nn as nn
import torch.nn.init as init

class TwoLayerNet(nn.Module):
    def __init__(self, D_in, H, D_out):
        super().__init__()

        self.linear1 = nn.Linear(D_in, H)
        self.relu = nn.ReLU()
        self.linear2 = nn.Linear(H, D_out)

        self._init_weights()

    def _init_weights(self):
        init.ones_(self.linear1.weight)
        if self.linear1.bias is not None:
            init.zeros_(self.linear1.bias, 0)
        init.kaiming_uniform_(self.linear2.weight, mode='fan_in', nonlinearity='relu')
        if self.linear2.bias is not None:
            init.constant_(self.linear2.bias, 0)

    def forward(self, x):
        h = self.linear1(x)
        h_relu = self.relu(h)
        y_pred = self.linear2(h_relu)
        return y_pred

dtype = torch.float32
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

model = TwoLayerNet(D_in, H, D_out)
model.to(dtype=dtype, device=device)

loss_fn = nn.MSELoss(reduction='sum')
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Dataset & DataLoaders

Dataset

- You can load your custom data by writing your own Dataset class.
- Why do we need it? (Lazy Loading)
 - Bad: Loading 50,000 images into RAM at once → Crash.
 - Good: Storing 50,000 filenames is cheap. We only load the actual image when requested.
- Implementation:
 - `__init__`
 - `__len__`
 - `__getitem__`

```
import torch
from torch.utils.data import Dataset

class MyDataset(Dataset):
    def __init__(self, size, length):
        """
        Setup the data source.
        In a production scenario, we usually store file paths or database handles here,
        rather than loading the entire dataset into RAM (Lazy Loading preparation).
        """
        self.len = size
        self.data = torch.randn(size, length)
        self.labels = torch.randint(0, 2, (size,))

    def __len__(self):
        """
        Returns the total cardinality (size) of the dataset.
        Required by DataLoader to calculate the number of batches per epoch.
        """
        return self.len

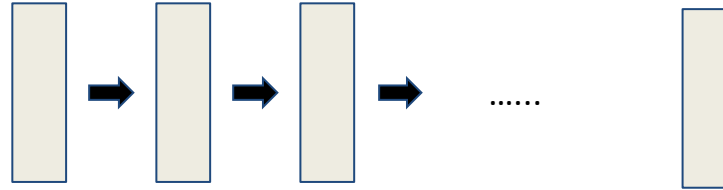
    def __getitem__(self, index):
        """
        Retrieves the sample at the specified index.
        Key logic for 'Lazy Loading' and 'On-the-fly Transforms' happens here.
        """
        return self.data[index], self.labels[index]
```

(Mini)batching

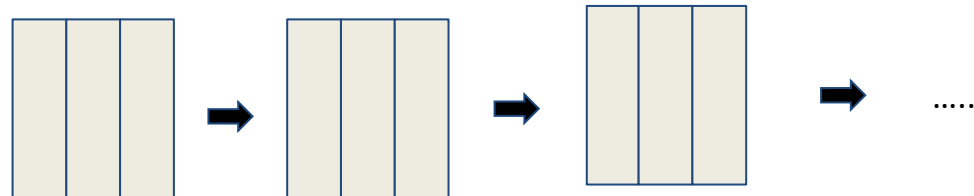
All your training data:



Standard for loop:



(Mini) batching:



DataLoader

- A DataLoader wraps a Dataset and provides minibatching, shuffling, multithreading, etc. for you

```
from torch.utils.data import DataLoader
dataset = MyDataset(size=1000, length=100)

loader = DataLoader(dataset=dataset,
                    batch_size=64,
                    shuffle=True,
                    num_workers=8)

# Loop over epochs
for epoch in range(100):
    for i, (batch_x, batch_y) in enumerate(loader):
        # Transfer to GPU
        batch_x, batch_y = batch_x.to(device), batch_y.to(device)

        # Model computations
        [...]
```

Complex Data

```
data = [  
    ("I", "am", "at", "USC", "!"), 1),  
    ("CSCI", "544", "is", "hard", "."), 0),  
    ("PyTorch", "is", "cool"), 1)  
]
```

- What if your data varies in length (e.g., text) or structure?
- The default DataLoader cannot automatically stack them, so it returns a raw list.
- However, models cannot handle lists; they require uniform, rectangular Tensors.

collate_fn

- A “Merge Function” used by the DataLoader.
- Padding:
 - In NLP, we usually need to face cases that requires “padding” so as to batch your data
 - Pad: `torch.nn.utils.rnn.pad_sequence`
 - Unpack:
`torch.nn.utils.rnn.pack_padded_sequence`

```
import torch
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence

raw_data = [
    ([ "I", "am", "at", "USC", "!" ], 1),
    ([ "CSCI", "544", "is", "hard", "." ], 0),
    ([ "PyTorch", "is", "cool" ], 1),
    ...
]

vocab = { "<PAD>": 0, "I": 1, "am": 2, "at": 3, "USC": 4, "!": 5,
          "CSCI": 6, "544": 7, "is": 8, "hard": 9, ".": 10,
          "PyTorch": 11, "cool": 12 }

class MyDataset(Dataset):
    def __init__(self, data, vocab):
        self.data = data
        self.vocab = vocab

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        words, label = self.data[idx]
        ids = [self.vocab[w] for w in words]
        return torch.tensor(ids), label

def padding_collate_fn(batch):
    inputs, labels = zip(*batch)
    padded_inputs = pad_sequence(inputs, batch_first=True, padding_value=0)
    labels = torch.tensor(labels)

    return padded_inputs, labels

loader = DataLoader(MyDataset(raw_data, vocab),
                    batch_size=2,
                    collate_fn=padding_collate_fn)
```

Summary

1. Prepare your data
 1. Write your own Dataset (inherit `torch.util.dataset`)
 2. (Write `collate_fn`)
2. Create your model
 1. A sequential module if your model is super easy and will not be reused, or
 2. A `nn.Module` module
3. Write the loop (how many epoch/steps) to train your model:
 1. Create a `DataLoader` that wraps the Dataset you provide
 2. Set an optimizer
 3. Set a loss for optimization
 4. For loop....
 1. Forward pass
 2. Zero-grad
 3. Backward pass => Get gradient
 4. Optimizer step to update your models' weight.

Tips on Implementation

Coding Tips

- Don't hardcode `.cuda()`
 - Always define a device object at the top and pass it around
- Fix seed
 - `random.seed(seed)`
 - `np.random.seed(seed)`
 - `torch.manual_seed(seed)`
- `train()` and `eval()`
 - Model may perform differently on training and validation loop
 - Dropout
 - BatchNorm

```
# Training Loop
model.train()
...

# Validation Loop
model.eval()
with torch.no_grad():
    ...
```

Coding Tips

- `item()` matters
 - **Bad:** `total_loss += loss`
 - `loss` is a Tensor attached to the entire Graph.
 - Keeping it alive keeps the whole history in memory, leading to OOM
 - **Fix:** `total_loss += loss.item()`
- Danger of in-place operations
 - `x += val` vs. `x = x + val`
 - Backward pass needs the original values to calculate gradients.
 - If you overwrite `x` in-place, the "history" is corrupted. PyTorch can't travel back!

Prepare Different Version of Data

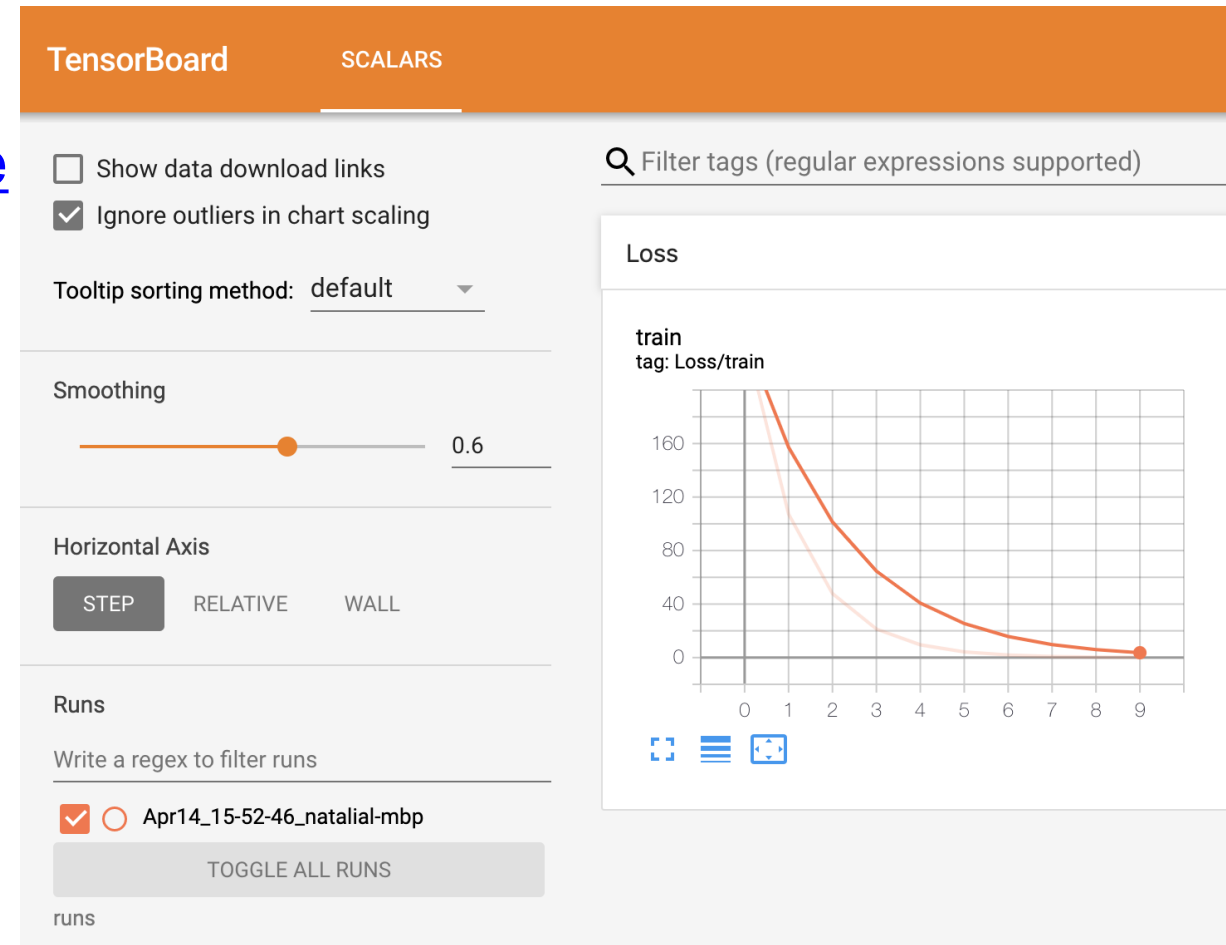
- Tiny-size: for debugging syntactic bug (especially for dynamic language, such as python)
- Small-size: check the behavior of the model
- Mid-size: for understanding model behavior and fast development
- Full-size: conduct final experiments

TensorBoard

- Installation:
https://pytorch.org/tutorials/recipes/recipes/tensorboard_with_pytorch.html
- Create a Summary Writer

```
from torch.utils.tensorboard import SummaryWriter  
writer = SummaryWriter()
```
- During Training/Dev/Test

```
writer.add_scalar("Loss/train", loss, epoch)
```



Acknowledgement

- Several materials from PyTorch Official Materials:
<https://pytorch.org/tutorials/>